Practical - 11

Aim:

Implement client-server communication using sockets in Python.

Q1: Create server.py and client.py file and create chat applications for continuous communication between both.

Server.py:

```
import socket
import time

hostname = input("Enter the hostname or IP address of the server: ")
port = int(input("Enter the port number to connect to: "))

ss = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
ss.bind((hostname, port))
ss.listen()
cs, addr = ss.accept()

print("Connection from: " + str(addr))

while True:
    msg = input("Server> ")
    cs.send(msg.encode())
    time.sleep(1)

    msg = cs.recv(1024).decode()
    if msg.lower() == "exit":
        print("Connection closed by the Client.")
        break
    print("Client> " + msg)

cs.close()
```

Output:

```
Vatsal@Vatsal-PC MINGW64 ~/Desktop/Sem4/P-PY/Practical-Code/Practical-11 (main)
$ py server.py
Enter the hostname or IP address of the server: localhost
Enter the port number to connect to: 12001
Connection from: ('127.0.0.1', 65217)
Server> hii From Server
Client> Hii From Client
Server>
```

Client.py:

```
import socket
import time

hostname = input("Enter the hostname or IP address of the server: ")
port = int(input("Enter the port number to connect to: "))

cs = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
ip = socket.gethostbyname(hostname)
cs.connect((ip, port))

print("Connection from: " + ip)
print("type 'exit' for exit.")

while True:
    msg = cs.recv(1024).decode()
    if msg.lower() == "exit":
        print("Connection closed by the Server.")
        break
    print("Server> " + msg)
    msg = input("Client> ")
    cs.send(msg.encode())

cs.close()
```

Output:

```
Vatsal@Vatsal-PC MINGW64 ~/Desktop/Sem4/P-PY/Practical-Code/Practical-11 (main)
$ py client.py
Enter the hostname or IP address of the server: localhost
Enter the port number to connect to: 12001
Connection from: 127.0.0.1
type 'exit' for exit.
Server> hii From Server
Client> Hii From Client
```

Q2: Write a python program to demonstrate usage of multi-threading.

Code:

```
from threading import Thread
import time


class th(Thread):
    def __init__(self, name):
        Thread.__init__(self)
```

```
        self.name = name

    def run(self):
        for i in range(5):
            print("Thread " + self.name + ": " + time.ctime(time.time()))


t1 = th("1")
t2 = th("2")
t1.start()
t2.start()
t1.join()
t2.join()
```

Output:

```
Vatsal@Vatsal-PC MINGW64 ~/Desktop/Sem4/P-PY/Practical-Code/Practical-11 (main)
$ py thread_test.py
Thread 1: Sun Apr 12 18:57:58 2026
Thread 2: Sun Apr 12 18:57:58 2026
Thread 1: Sun Apr 12 18:57:58 2026
Thread 2: Sun Apr 12 18:57:58 2026
Thread 2: Sun Apr 12 18:57:58 2026
Thread 1: Sun Apr 12 18:57:58 2026
Thread 2: Sun Apr 12 18:57:58 2026
Thread 1: Sun Apr 12 18:57:58 2026
Thread 1: Sun Apr 12 18:57:58 2026
Thread 2: Sun Apr 12 18:57:58 2026
```

Q3: Write a python code to send email by attaching .jpeg file as a content.

Code:

```
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from email.mime.application import MIMEApplication

sender = 'vatsalpatel0609@gmail.com'
receiver = '24012011142@gnu.ac.in'

msg = MIMEMultipart()
msg['From'] = sender
msg['To'] = receiver
msg['Subject'] = 'Sending Image From Python'

body = "Hello, this is a test email with an image attachment sent from Python."
msg.attach(MIMEText(body, 'plain'))

file = open('Python.png' , 'rb')
attachment = MIMEApplication(file.read(), name='Python.png')
attachment['Content-Disposition'] = 'attachment; filename="Python.png"'
```

```python
msg.attach(attachment)

# 4. Connect to SMTP server and send the email
smtpob = smtplib.SMTP('smtp.gmail.com', 587)
smtpob.starttls()
# Use your 16-character App Password here
smtpob.login(sender, "Your_App_Password")

smtpob.sendmail(sender, receiver, msg.as_string())
smtpob.quit()

print("Email sent successfully!")
```

Output:

```
Vatsal@Vatsal-PC MINGW64 ~/Desktop/Sem4/P-PY/Practical-Code/Practical-11 (main)
$ py send_mail.py
Email sent successfully!
```